



Path With Given Sequence (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
- Time complexity
- Space complexity

Problem Statement

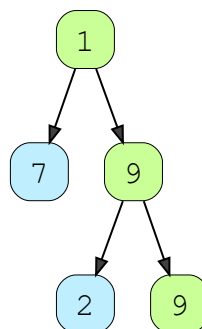
Given a binary tree and a number sequence, find if the sequence is present as a root-to-leaf path in the given tree.

Example 1:

Sequence: [1, 9, 9]

Output: true

Explanation: The tree has a path 1 -> 9 -> 9.



Example 2:

Sequence: [1, 0, 7]

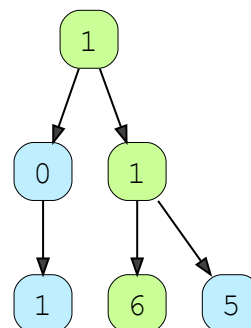
Output: false

Explanation: The tree does not have a path 1 -> 0 -> 7.

Sequence: [1, 1, 6]

Output: true

Explanation: The tree has a path 1 -> 1 -> 6.



Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```

1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7
8      TreeNode(int x) {
9          val = x;
10     }
11 };
12
13 class PathWithGivenSequence {
14     public static boolean findPath(TreeNode root, int[] sequence) {
15         // TODO: Write your code here
16         return false;
17     }
18
19     public static void main(String[] args) {
20         TreeNode root = new TreeNode(1);
21         root.left = new TreeNode(0);
22         root.right = new TreeNode(1);
23         root.left.left = new TreeNode(1);
24         root.right.left = new TreeNode(6);
25         root.right.right = new TreeNode(5);

```



```

29     }
30 }
31

```

Run

Save

Reset



Solution

This problem follows the [Binary Tree Path Sum](#) pattern. We can follow the same **DFS** approach and additionally, track the element of the given sequence that we should match with the current node. Also, we can return **false** as soon as we find a mismatch between the sequence and the node value.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7
8      TreeNode(int x) {
9          val = x;
10     }
11 };
12
13 class PathWithGivenSequence {
14     public static boolean findPath(TreeNode root, int[] sequence) {
15         // TODO: Write your code here
16         return false;
17     }
18
19     public static void main(String[] args) {
20         TreeNode root = new TreeNode(1);
21         root.left = new TreeNode(0);
22         root.right = new TreeNode(1);
23         root.left.left = new TreeNode(1);
24         root.right.left = new TreeNode(6);
25         root.right.right = new TreeNode(5);

```

```

13 class PathWithGivenSequence {
14     public static boolean findPath(TreeNode root, int[] sequence) {
15         if (root == null)
16             return sequence.length == 0;
17
18         return findPathRecursive(root, sequence, 0);
19     }
20
21     private static boolean findPathRecursive(TreeNode currentNode, int[] sequence, int sequenceIndex) {
22
23         if (currentNode == null)
24             return false;
25
26         if (sequenceIndex >= sequence.length || currentNode.val != sequence[sequenceIndex])
27             return false;
28
29         // if the current node is a leaf, add it is the end of the sequence, we have found a path!
30         if (currentNode.left == null && currentNode.right == null && sequenceIndex == sequence.length - 1)
31             return true;
32
33         // recursively call to traverse the left and right sub-tree
34         // return true if any of the two recursive call return true
35         return findPathRecursive(currentNode.left, sequence, sequenceIndex + 1)
36             || findPathRecursive(currentNode.right, sequence, sequenceIndex + 1);
37     }
38
39     public static void main(String[] args) {
40         TreeNode root = new TreeNode(1);
41         root.left = new TreeNode(0);
42         root.right = new TreeNode(1);
43         root.left.left = new TreeNode(1);
44         root.right.left = new TreeNode(6);
45         root.right.right = new TreeNode(5);
46
47         System.out.println("Tree has path sequence: " + PathWithGivenSequence.findPath(root, new int[] { 1, 0, 7 }));
48         System.out.println("Tree has path sequence: " + PathWithGivenSequence.findPath(root, new int[] { 1, 1, 6 }));
49     }
50 }
51

```

Run

Save

Reset

⌵

Time complexity

The time complexity of the above algorithm is $O(N)$, where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space complexity

The space complexity of the above algorithm will be $O(N)$ in the worst case. This space will be used to store the recursion stack. The worst case will happen when the given tree is a linked list (i.e., every node has only one child).

[< Back](#)
☒ Mark As Completed

[Next >](#)

Sum of Path Numbers (medium)

Count Paths for a Sum (med) 

